UNIVERSITÀ di VERONA
Dipartimento di INFORMATICA

Discrete Optimization and Decision Making

Introduction to Gurobi

Master's degrees in Data Science - Artificial Intelligence - Mathematics
Academic Year 2024/2025

Romeo Rizzi (Dept. of Computer Science, Univ. of Verona)
Roberto Zanotti (Dept. of Clinical and Experimental Sciences, Univ. of Brescia)

Introduction

Gurobi Optimizer

- Current state-of-the-art **commercial software**, written in C, dedicated to the optimization of mathematical problems of different types (e.g., LP, MILP, MIQP).
- Developed by Zonghao **Gu** and Edward **Ro**thberg (which led the IBM CPLEX development team for a decade), together with Robert **Bi**xby (the founder of CPLEX).
- Widely used in **industry and academia**.

Features

- Implementation of:

  - LP Solver: primal and dual simplex algorithms, parallel barrier algorithm with crossover, concurrent optimization, and sifting algorithm
  - QP Solver: simplex and parallel barrier algorithms
  - QCP Solver: parallel SOCP barrier algorithm
  - MIP Solver: deterministic, parallel branch-and-cut, non-traditional tree-of-trees search, multiple default heuristics, solution improvement, cutting planes, and symmetry detection

- Standard MIP **cutting-plane routines** and also new routines developed on purpose and not known to the public.
- **Interfaces** for object oriented languages (e.g., C++, Java, Python) and matrix-oriented languages (e.g., C, MATLAB, R).
- Links to standard **modelling languages** (e.g., AMPL, MPL).
- Extremely robust code and **parallelism**, designed to exploit multi-core processors out of the box.

In a nutshell

How to install Gurobi in a nutshell

1. Go to `www.gurobi.com` and create an account by using your institutional email.
2. Download and install the appropriate Gurobi Optimizer version according to your operating system.
3. Go to your license page in the website and, from a command/terminal prompt, execute the command you will find there:

```
grbgetkey license-code
```

4. You have to be connected to the university network to be able to activate your license.

A first example in Python

The example

We will use a simple Python example in order to present the main features of the Gurobi Python Interface.

In this example we will:

- build a model;
- optimize it;
- query some information about it and the solution obtained.

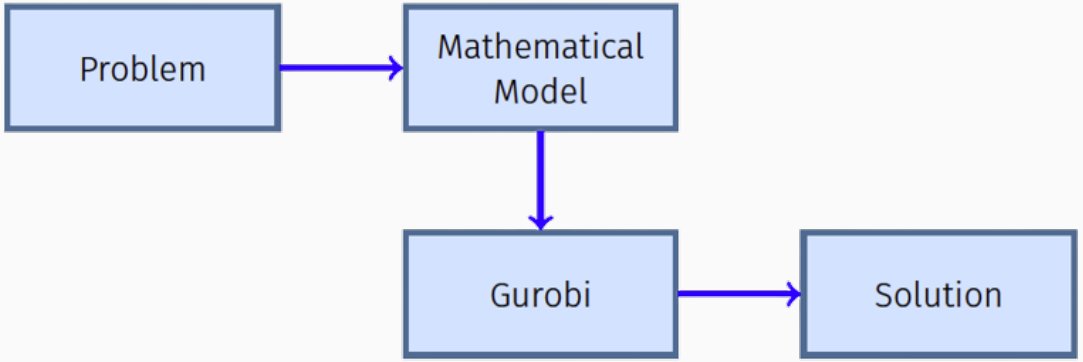
We will optimize the following model:

$$\begin{aligned} \max \quad & x + y + 2z \\ & x + 2y + 3z \leq 4 \\ & x + y \geq 1 \\ & x, y, z \in \{0, 1\} \end{aligned}$$

You can find the full example on the Moodle Community: `MIPExample.py`.

Building the model

Gurobi as a black-box solver:



Solution: assigning a value to each one of the problem variables.

However, a solution might not be optimal, because:

- it does not exist;
- Gurobi requires too much time and/or memory.

Gurobi provides several information about the solution process and the solution itself.

The environment

Our example begins by importing the Gurobi classes:

```
import gurobipy as gp
from gurobipy import GRB
```

Then we create the environment and start it:

```
env = gp.Env()
env.start()
```

- You can specify the name of the log file (or an empty string if you do not want a log).
- You can provide a dict with parameter-value pairs to quickly set up the env.
- We will always need an environment to optimize a model!

Environment parameters

It is sometimes useful to change some environment parameters, such as:

```
env.setParam("Threads", 4)
env.setParam("Presolve", 2)
env.setParam("MIPGap", 1e-4)
env.setParam("TimeLimit", 600)
```

You can find the complete list of parameters here: `https://www.gurobi.com/documentation/11.0/refman/parameter_descriptions.html`.

Environment parameters – Number of threads

```
env.setParam("Threads", 4)
```

- The Threads parameter determine how many threads/cores Gurobi will use while optimizing a model in this environment.
- It is useful when multiple models from different environments are solved simultaneously.
- The default value is 0 (i.e., all cores are exploited).

Environment parameters – Presolve

```
env.setParam("Presolve", 2)
```

- The Presolve parameter controls the presolve level:
  - -1 corresponds to an automatic setting;
  - 0 off;
  - 1 conservative;
  - 2 aggressive.
- A more aggressive usage of presolve takes more time, but can sometimes lead to a model significantly faster to solve.
- The default value is -1.

Environment parameters – MIPGap

```
env.setParam("MIPGap", 1e-4)
```

- $MIPGap = \frac{|Incumbent - BestBound|}{|Incumbent|}$.
- If $MIPGap < 10^{-4}$ (the value specified in the command), the optimization process stops and Gurobi returns the Incumbent solution as if it were optimal.
- To avoid missing potentially optimal solutions, it is better to set the MIPGap parameter to a sufficiently low value, typically

$$\leq \frac{1}{LP_Solution_Value}.$$

Environment parameters – TimeLimit

```
env.setParam("TimeLimit", 600)
```

- The TimeLimit parameter limits the total time spent finding an optimal solution for a model.
- It is expressed in seconds.
- The default value is ∞

The model

Once we have an environment, we can create a model as follows:

```
model = gp.Model("MIP-Example", env=env)
```

A Gurobi Model holds a single optimization problem. It consists of a set of variables, a set of constraints, an objective function and all their attributes.

- The first parameter of the Model function specifies the name of our problem.
- The second parameter is the environment the model is associated with.

With the above code we create an empty model object and we will later add all the required information.

Adding variables

The next step is to add variables to the model:

```
x = model.addVar(0, 1, 0, GRB.BINARY, "x")
y = model.addVar(0, 1, 0, GRB.BINARY, "y")
z = model.addVar(0, 1, 0, GRB.BINARY, "z")
```

- A variable is always associated with a particular model.
- Variables are added through the addVar() method on a Model object.
- Arguments:
  - the first one is the variable **lower bound, lb**;
  - the second one is the variable **upper bound, ub**;
  - the third one is the variable **linear objective coefficient, obj** (here it is 0 because we will set the objective function later);
  - the fourth argument is the variable **type, vtype** (`GRB.BINARY`, `GRB.INTEGER`, or `GRB.CONTINUOUS`);
  - the last one is the variable **name, name**.

Objective function

The next step is to set the optimization objective:

```
# Set objective:  maximize x + y + 2z
model.setObjective(x + y + 2*z, GRB.MAXIMIZE)
```

- The first argument is the expression representing the objective function
- The second argument is the direction of the optimization

Constraints

We now add the constraints to the model:

```
#Add constraint:  x + 2y + 3z <= 4
model.addConstr(x + 2*y + 3*z <= 4, name="c0")
#Add constraint:  x + y >= 1
model.addConstr(x + y >= 1, name="c1")
```

- As for the variables, constraints are always associated with a model.
- We use the addConstr() method and pass some **arguments**:
  - the first one is the expression representing the constraint;
  - the second one is the constraint name;

Optimization

Now that the model has been built, the next step is to optimize it:

```
model.optimize()
```

- This method performs the optimization of the model and populates several internal model attributes.
- The Status attribute of the model indicates the state of the optimization process when it ended.
- Some of the most common values of the Status attribute are the following:
  - `GRB.OPTIMAL`
  - `GRB.INFEASIBLE`
  - `GRB.TIME_LIMIT`

Reporting results

Once the optimization is complete, we can query the values of the attributes:

```
x.VarName
x.getAttr("X")
model.objVal
```

- Attribute VarName contains the name of the variable.
- Attribute X contains the solution value of the variable.
- The model attribute objVal contains the objective value for the current solution.

Solving the continuous relaxation

It is often useful to solve the continuous relaxation of the model:

```
relaxed = model.relax()
relaxed.optimize()
x.getAttr("RC")
```

- To solve the continuous relaxation of the current MIP model, we first create the relaxed model by calling the relax() method of the original Model object.
- After the relaxation is solved, other attributes can be queried (e.g., the RC variable attribute, which contains the reduced costs of the variable).

Cleaning up in the end

You need to get rid of the model and the environment once you do not need them anymore:

```
model.dispose()
env.dispose()
```

- These methods free the resources associated with the model and the environment.
- Garbage collection would claim these resources eventually, but if your program does not exit immediately after performing the optimization, it is best to free them explicitly.

The log

```
Academic license - for non-commercial use only
Read MPS format model from file C:\Users\Roberto\workspace\Progetto2019\instances\rococoC10-001000.mps
Reading time = 0.02 seconds
rococoC10-001000: 1293 rows, 3117 columns, 11751 nonzeros
Optimize a model with 1293 rows, 3117 columns and 11751 nonzeros
Coefficient statistics:
  Matrix range     [1e+00, 3e+04]
  Objective range  [1e+00, 1e+00]
  Bounds range     [1e+00, 3e+05]
  RHS range        [1e+00, 1e+00]
Presolve removed 750 rows and 675 columns
Presolve time: 0.00s
Presolved: 543 rows, 2442 columns, 6806 nonzeros

Iteration    Objective       Primal Inf.    Dual Inf.      Time
       0    0.0000000e+00   2.022254e+03   0.000000e+00      0s
    1494    7.5152710e+03   0.000000e+00   0.000000e+00      0s

Solved in 1494 iterations and 0.04 seconds
Optimal objective  7.515271029e+03
```

Exporting the model or some information

Sometimes it is useful to save the model in a standard format, in order to directly read it later or to print some information:

```
model.write("MIP-example.mps")
```

The only argument is the filename, with its corresponding suffix. Valid suffixes:

- `.mps`, `.rew` `.lp` or `.rlp` for writing the model itself;
- `.dua` or `.dlp` for the dualized model (only pure LP);
- `.ilp` for writing just the IIS associated with an infeasible model (obtained through `model.computeIIS()`);
- `.sol` for writing the solution selected by the SolutionNumber parameter;
- `.mst` for writing a start vector.

For more details: `https://www.gurobi.com/documentation/11.0/refman/model_file_formats.html`.

Importing the model or or a solution

Sometimes it is useful to load a previously saved model:

```
from gurobipy import read
model = read("MIP-Example.lp", env)
```

The only argument is the filename, with its corresponding suffix.

You can also read a solution:

```
model.read("MIP-Example.sol")
```